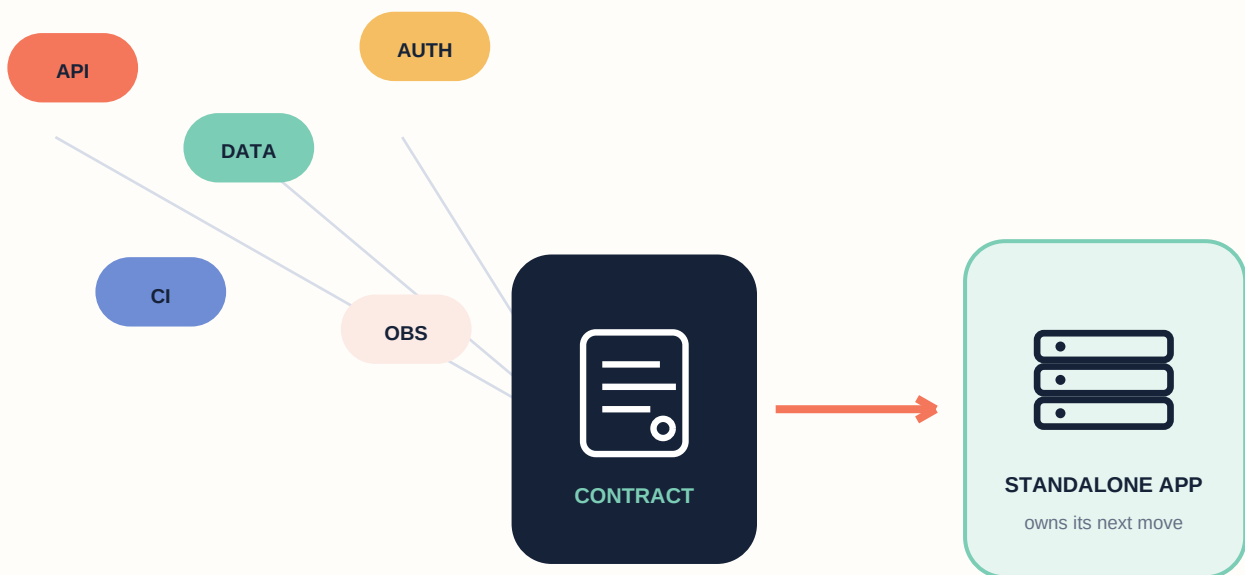




.NET

TEMPLATE CONTRACT

Blueprint. Not framework.

Designing an option-driven .NET service template with explicit contracts, isolated output, and honest release gates.

The clever part is not how much code the template can add. It is how clearly the template says what it will add, what it will reject, and when it is done.

The contract is visible - and countable.

The current repository turns variation into a machine-readable agreement, then checks generated output at milestone boundaries.

19

explicit options

Runtime, API, data, security, integration, testing, and delivery choices live in one versioned contract.

13

compatibility rules

6 normalize, 5 require, 2 warn. Invalid intent is corrected, blocked, or surfaced before generation.

5

milestone test entrypoints

M4 through M8 cover generation, data, capabilities, production readiness, and adoption checks.

2

target frameworks

.NET 8 and .NET 10 are represented in the option contract and framework-specific generation paths.

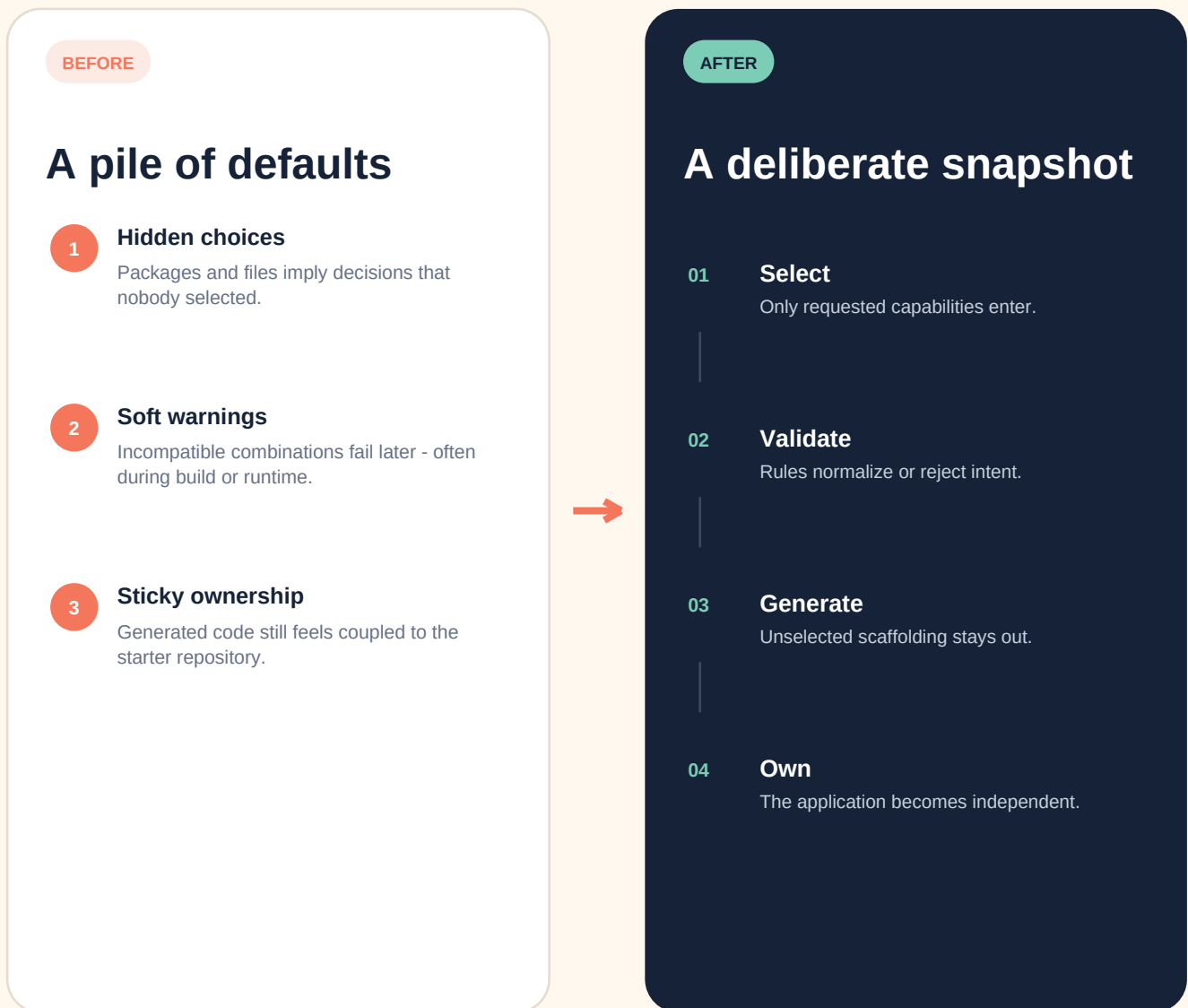
THE RESULT

A choice is no longer a README suggestion. It becomes input, a rule, generated output, and evidence.

Counts verified from config/template-options.json and tests/m4-m8

From starter-code folklore to a contract.

The shift is organizational as much as technical: decisions become explicit before files appear.



The template ends at generation. The generated application begins there.

Five moves. One standalone service.

The template is a creation-time tool, not a runtime dependency or upgrade mechanism.



CAPABILITIES INCLUDED ONLY WHEN SELECTED

- | | |
|--|---|
| <ul style="list-style-type: none">API
Controllers / Minimal / None | <ul style="list-style-type: none">DATA
EF Core / PostgreSQL / SQL Server / SQLite |
| <ul style="list-style-type: none">SECURITY
JWT / OIDC / Policy | <ul style="list-style-type: none">INTEGRATION
HTTP / Messaging / Hybrid |
| <ul style="list-style-type: none">DELIVERY
Containers / GitHub Actions / Azure DevOps | <ul style="list-style-type: none">OPERATIONS
Health / OpenTelemetry / Cache / Jobs |

EXPLICITLY OUTSIDE THE TARGET

Desktop and client UI, Razor Pages, MVC UI, games, Azure Functions, Kubernetes provisioning, and legacy .NET Framework applications.

A green build is not a broker test.

The repository separates what generates and compiles from what has durable infrastructure evidence.

generation + build

!=

**integration
evidence**

Compilation proves structure. Production claims need behavior under real infrastructure, failure, retry, and recovery.

CURRENTLY PRODUCTION-CAPABLE

API
EF Core persistence
Health checks
OpenTelemetry wiring
Container options

STILL NEEDS DURABLE EVIDENCE

Broker adapters
Outbox and inbox-outbox dispatch
Hangfire storage and recovery
Telemetry export assertion

LOCAL CONSTRAINT, HONESTLY ROUTED

.NET 10 and Docker smoke checks remain CI-owned on this machine. The gate stays; only its execution venue changes.

Five rules for templates that age well.

A durable template behaves less like a code dump and more like a small, testable product contract.

01

Contract before content

Make every supported choice, default, and incompatibility explicit before generating files.

02

Exclude what was not chosen

Optional capability means absent code and packages - not dormant scaffolding.

03

End cleanly

Generated output must build and evolve without referencing the template repository.

04

Match proof to the claim

Generation tests prove shape; runtime and infrastructure tests prove behavior.

05

Keep upgrades application-owned

Do not regenerate over a live service. Adopt changes through normal review and migration.

THE SHORT VERSION

Build a funnel, not a cage: constrain the creation decision, then let the application go.